

Parámetros de Verificación de la Información en Sistemas de Información (SI)

1. Contenido (El Modelo de Datos)

- En los SI: Se refiere a la estructura lógica de la base de datos y a la completitud de los registros (entidades y atributos).
- Aplicación práctica: Si se diseña un simulador interactivo para la evaluación de desempeño, el "contenido" implica asegurar que el modelo de datos capture todas las variables necesarias (competencias, metas, retroalimentación) sin omitir campos críticos. Un SI con contenido deficiente genera reportes vacíos o análisis superficiales.

2. Vigencia (Sincronización y Ciclo de Vida del Dato)

- En los SI: Está directamente relacionado con la actualización en tiempo real, las marcas de tiempo (timestamps) y las políticas de retención de datos.
- Aplicación práctica: Un aplicativo en Vanilla JS que calcule provisiones de nómina debe consultar las tasas y normativas del año en curso. Si el sistema almacena parámetros del año pasado en su memoria caché o base de datos y no alerta sobre su caducidad, la vigencia falla, provocando errores en toda la operación financiera.

3. Pertinencia (Filtrado y Consultas)

- En los SI: Se traduce en el diseño eficiente de consultas (queries) y en la arquitectura de la información. Un sistema pertinente entrega exactamente lo que el usuario necesita en su contexto actual, evitando la sobrecarga de datos.
- Aplicación práctica: Al construir un dashboard para coordinadores, el sistema debe filtrar mediante sentencias lógicas y mostrar únicamente los indicadores clave de su área específica. Mostrarle a un instructor el flujo de caja de toda la institución en lugar del avance de sus aprendices es un fallo de pertinencia en la interfaz.

4. Oportunidad (Latencia y Disponibilidad)

- En los SI: Tiene que ver con los tiempos de respuesta del servidor, la latencia de la red, el uptime (tiempo en línea) y el procesamiento asíncrono.
- Aplicación práctica: De nada sirve un sistema que genera alertas de ausentismo si el procesamiento de los datos tarda tres días. La oportunidad exige que el SI procese y muestre la información en el momento exacto en que el área de talento humano puede intervenir.

5. Coherencia (Integridad Referencial y Lógica)

- En los SI: A nivel de backend, es la integridad referencial (evitar datos huérfanos o contradictorios en distintas tablas). A nivel de frontend, es la consistencia en la interfaz de usuario (UI) y la lógica de validación.
- Aplicación práctica: Un sistema pierde coherencia si en el módulo de contratación un perfil aparece como "Activo", pero en el módulo de Microsoft Planner integrado aparece como "Inactivo" o "Retirado". La coherencia obliga a que todas las partes del sistema estén sincronizadas y hablen el mismo idioma.

6. Soporte (Trazabilidad y Logs de Auditoría)

- En los SI: Corresponde a los registros de actividad (logs), el historial de versiones y las copias de seguridad.
- Aplicación práctica: Si un usuario cuestiona por qué se rechazó una solicitud en una plataforma web, el SI debe tener un soporte técnico: un registro en la base de datos que indique exactamente qué usuario, en qué fecha y bajo qué criterio modificó ese estado.

7. Fuente (Procedencia y Consumo de APIs)

- En los SI: Se refiere a los orígenes de los datos. Puede ser entrada manual de usuarios, integración con sistemas heredados (legacy) o consumo de servicios externos mediante APIs.
- Aplicación práctica: Al integrar herramientas con inteligencia artificial generativa, la fuente de los datos que alimentan el modelo debe ser confiable. Si el SI extrae datos de una API externa no oficial o no verificada, toda la información procesada posteriormente queda comprometida.

8. Cálculos (Algoritmos y Lógica de Negocio)

- En los SI: Son las funciones matemáticas, estadísticas y algoritmos codificados en el núcleo del sistema.
- Aplicación práctica: En una Single-Page Application (SPA) diseñada para organizar torneos o dinámicas gamificadas, la función en JavaScript que calcula los emparejamientos, puntajes o estadísticas debe ser matemáticamente infalible. Un error en un bucle o en una variable arruina la integridad de los resultados.

9. Recursos (Eficiencia Computacional e Infraestructura)

- En los SI: Evalúa el consumo de memoria, ancho de banda, capacidad de procesamiento y costos de alojamiento (hosting).
- Aplicación práctica: Validar si una aplicación HTML/CSS requiere un servidor dedicado costoso o si su código está lo suficientemente optimizado para ejecutarse en entornos más ligeros (como GitHub Pages o Netlify) sin degradar la experiencia del usuario.

10. Alcance (Roles, Permisos y Escalabilidad)

- En los SI: Determina los límites del sistema mediante el control de acceso basado en roles (RBAC) y la arquitectura (local, regional, global).
- Aplicación práctica: El alcance define si un módulo de software está diseñado para gestionar 50 usuarios de un solo centro de formación, o si su arquitectura es lo suficientemente robusta para escalar a miles de usuarios a nivel nacional con perfiles administrativos distintos.

11. Difusión (Interfaces, APIs de salida y Reportes)

- En los SI: Es la capa de presentación (Frontend). Cómo el sistema empaqueta y entrega la información procesada para que sea consumible por el usuario final u otros sistemas.
- Aplicación práctica: Un sistema excelente en su backend falla en la difusión si su diseño visual (usando herramientas como Tailwind CSS) no es responsivo, si las notificaciones de Microsoft Teams no llegan con el formato adecuado, o si no permite exportar un reporte estructurado en PDF para las reuniones de equipo.

Para validar que la información dentro de un Sistema de Información (SI) cumple a cabalidad con estos once parámetros, debes aplicar técnicas de auditoría de datos, pruebas de software (Quality Assurance o QA) y validaciones de interfaz.

Ferney, cuando estés evaluando las aplicaciones web que desarrollas o revisando los proyectos de tus aprendices, puedes aplicar esta metodología práctica para cada parámetro:

- **Contenido (Auditoría de Esquemas y Requerimientos):** Se valida cruzando el diseño de la base de datos (o la estructura de tus objetos JSON) con el documento de requerimientos inicial. *Acción:* Crea una matriz de trazabilidad. Si el sistema es de contratación, verifica campo por campo que el formulario de registro incluya todo lo exigido por la ley y que no queden valores en blanco o nulos donde debería haber información obligatoria.
- **Vigencia (Pruebas de Caducidad y Timestamps):** Se valida inspeccionando las marcas de tiempo de los registros y la lógica de actualización. *Acción:* Revisa el código en JavaScript para asegurar que los parámetros variables (como el salario mínimo o tasas de impuestos) no estén "quemados" (hardcoded) de forma permanente. Introduce fechas pasadas en el entorno de pruebas y verifica si el sistema arroja alertas de "información desactualizada".
- **Pertinencia (Pruebas de Usabilidad y UX):** Se valida mediante sesiones de prueba con el usuario final (User Acceptance Testing - UAT). *Acción:* Siéntate con un coordinador de talento humano, pídele que busque una competencia específica de un empleado y observa si el sistema le muestra datos irrelevantes. Revisa que las sentencias de filtrado en el *frontend* estén limpias y enfocadas.

- **Oportunidad (Monitoreo de Rendimiento y Latencia):** Se valida midiendo los tiempos de carga y procesamiento. *Acción:* Utiliza la pestaña *Network* de las DevTools del navegador para medir cuánto tarda una petición (fetch) en devolver la lista de asistencia. Si un reporte clave tarda minutos en generarse, el sistema falla en oportunidad.
- **Coherencia (Pruebas de Integridad Referencial y Flujo Cruzado):** Se valida simulando el ciclo de vida completo de un dato a través de todos los módulos. *Acción:* Cambia el estado de un perfil a "Inactivo" en el módulo principal. Luego, navega a los módulos de nómina, evaluación y planeación para comprobar que en todos aparezca como "Inactivo" y no permita asignarle nuevas tareas.
- **Soporte (Revisión de Logs y Trazabilidad):** Se valida ejecutando transacciones críticas y auditando el registro de eventos del sistema. *Acción:* Modifica una nota o un indicador de desempeño y luego busca en la base de datos el *log* de auditoría. Si el sistema no puede responder "qué usuario hizo el cambio, desde qué IP y a qué hora", carece de soporte.
- **Fuente (Auditoría de APIs y Validaciones de Entrada):** Se valida rastreando el origen de los flujos de datos. *Acción:* Revisa la URL de los *endpoints* que consume tu aplicación. Asegúrate de que provengan de dominios oficiales mediante HTTPS. Para la entrada manual, verifica que existan validaciones estrictas en los formularios (tipos de datos, longitudes) para evitar que fuentes humanas ingresen basura al sistema.
- **Cálculos (Pruebas Unitarias o Unit Testing):** Se valida aislando las funciones matemáticas y sometiénolas a casos de prueba extremos (*edge cases*). *Acción:* Si tienes una función en Vanilla JS que calcula las horas extras, ingresa valores atípicos (cero horas, horas negativas, horas máximas legales) y compara el resultado arrojado por el sistema con un cálculo manual hecho en Excel.
- **Recursos (Pruebas de Estrés y Optimización):** Se valida monitoreando el consumo de hardware y red durante la ejecución de la app. *Acción:* Evalúa el peso de tu código CSS y JavaScript. Prueba la aplicación en computadoras con poca memoria RAM o conexiones a internet lentas para verificar que la arquitectura elegida (por ejemplo, cargarla desde GitHub Pages) soporta la carga sin bloquear el navegador.
- **Alcance (Matriz de Roles y Permisos):** Se valida mediante pruebas de intrusión de perfiles. *Acción:* Inicia sesión con credenciales de un usuario estándar (como un aprendiz o empleado base) e intenta acceder mediante la URL directa a los paneles de configuración exclusivos del administrador. El sistema debe bloquear el acceso y redirigir al usuario inmediatamente.
- **Difusión (Pruebas de Interfaz y Exportación):** Se valida comprobando que la salida de la información sea legible, accesible y exportable. *Acción:* Redimensiona la ventana del navegador para asegurar que las clases de Tailwind CSS mantienen el reporte visualmente organizado en móviles y tablets. Además, genera una exportación en PDF o Excel y revisa que la estructura de los datos no se rompa y sea lista para presentar en una reunión.